Academic Year	Module	Assessment Number	Assessment Type
S26	Artificial Intelligence and Machine Learning	01	NLP Text Classification

Hate speech Detection using RNN, LSTM,

and

LSTM with word2Vec Embeddings

Student Id : 2431895

Student Name : Saval Kaini

Module Leader : Mr. Siman Giri

Tutor : Mr. Bijaya Ghimire

Submitted on : 05-10-2026

Abstract

This work explores how machines can spot hateful words by using smart reading methods built on deep learning. Its main goal? Sorting written content into two groups - one that spreads hostility, another that does not. As digital chats multiply across websites and apps, picking out toxic expressions turns more urgent every day.

Getting there means using different deep learning setups, such as basic RNNs along with LSTMs. Before anything runs, text gets cleaned up - letters turned small, marks stripped out, empty words dropped, pieces split apart, sequences stretched evenly. After that comes word embedding, turning words into number groups a model can actually work with.

What stands out is how each model behaves when tested against measures like accuracy, F1-score, recall, precision, along with visual loss patterns and confusion outputs. It turns out the LSTM handles extended text relationships better than the basic RNN, showing stronger outcomes across trials.

A single test shows how well deep networks can spot hateful comments while boosting tools that sort content automatically. Moving ahead, trying models built on transformers - such as BERT or RoBERTa - might lift accuracy even more.

1. Table of Contents

1. Introduction 4
2. Dataset 4
   2.1 Dataset Features 5
3. Data Preprocessing 5
   3.1 Text Normalization 5
   3.2 Stopword Removal 5
   3.3 Tokenization 5
   3.4 Padding 6
   3.5 Train-Test Split 6
4. Methodology 6
5. Model Architectures 7
   5.1 Model 1: Simple RNN 7
   5.2 Model 2: LSTM 7
   5.3 Model 3: LSTM with Pretrained Word Embeddings 8
6. Loss Function and Optimizer 9
   6.1 Loss Function 9
   6.2 Optimizer 9
7. Hyperparameters 9
8. Experiments and Results 10
   8.1 Training and Validation Accuracy 10
   8.2 Training and Validation Loss 11
9. RNN vs LSTM Performance 11
10. Model Evaluation 12
11. Final Comparision of All Models 16
12. Conclusion and Future Work 17
   12.1 Key Findings 18
   12.2 Limitations 18
   12.3 Future Improvements 19
13. Appendix 19

1. Introduction

Out here, social media lets harsh words spread fast. When people face nasty messages, it often leads to unfair treatment or even physical harm. Sorting through endless posts by hand takes too long, wears anyone down. Because of that, tools that spot cruelty without help are becoming essential for safer spaces online.

Text analysis using computer systems has shown strong results lately. Instead of older methods, neural network designs now handle understanding written words better. One approach uses loops inside the system to follow word sequences over time. These setups remember earlier parts of sentences while processing new ones. Because of that, spotting harmful expressions becomes more accurate. The work here sorts messages by whether they contain offensive intent or not. Models like LSTM help catch patterns regular tools miss.

One way to look at it is through how RNNs stack up against LSTMs when handling sequences. Memory features seem to play a big role in making sense of written language, that much becomes clear. What stands out just as much is the groundwork before modeling begins. Things like breaking text into pieces matter more than they might first appear. Getting rid of common filler words helps too. So does lining up inputs to equal lengths, which keeps things running smoothly later on.

2. Dataset

This project works with a collection of written remarks pulled from digital spaces - comments or posts identified as hate speech. Some entries are tagged clearly, showing what kind fits where

- Hate Speech
- Offensive Language
- Neither

Into the workspace came the dataset, pulled in through Pandas. Missing pieces showed up first when checked. Then it looked at repeated lines - found some too. Distribution across groups followed next, revealing how each class lined up.

2.1 Dataset Features

The dataset mainly contains the following columns:

- Text / Tweet
- Label / Class

The text column contains user-generated content, while the label column represents the category of the text.

3. Data Preprocessing

Most times, cleaning up text comes before any real work in language apps. Messy inputs full of odd marks or random characters tend to throw off how well systems understand meaning. Fixing this problem started with a few preparation steps taken ahead of time.

3.1 Text Normalization

- Converted all text to lowercase.
- Removed punctuation marks.
- Removed URLs and HTML tags.
- Removed special characters and numbers.
- Removed extra whitespace.

3.2 Stopword Removal

- Stopword Removal: Common English stopwords were removed.
- Method: NLTK stopword library was used.

3.3 Tokenization

Tokenization transforms text sentences into sequences of words or numerical values.

Example:

"I hate this person" → [12, 45, 9, 88]

The Keras Tokenizer was used to convert words into numerical sequences.

3.4 Padding

Given that neural networks necessitate input sequences of equal length, the pad_sequences function was employed to pad shorter sequences with zeros.

3.5 Train-Test Split

The dataset was divided into training and testing sets:

- Training Set: 80%
- Testing Set: 20%

4. Methodology

The overall methodology of this project includes the following steps:

1. Data Collection
2. Data Cleaning and Preprocessing
3. Tokenization and Padding
4. Model Building
5. Model Training
6. Model Evaluation
7. Comparison of Models

- The project utilized three prominent deep learning architectures: a simple recurrent neural network (RNN), long short-term memory (LSTM), and LSTM with pretrained word embeddings.

5. Model Architectures

5.1 Model 1: Simple RNN

The first model implemented was a Simple Recurrent Neural Network (RNN).

The architecture consists of:

- Embedding Layer
- Simple RNN Layer
- Dense Output Layer

Out of each word, the embedding layer builds thick vector shapes. Moving through time, sequences take form inside the RNN layer, catching context as they go. Near the end, predictions come alive - shaped by a dense layer using sigmoid to decide the class.

Advantages of RNN

- Handles sequential text data
- Learns contextual information
- Simple architecture

Limitations of RNN

- Struggles with long-term dependencies
- Suffers from vanishing gradient problem

5.2 Model 2: LSTM

The second model implemented was Long Short-Term Memory (LSTM).

The architecture includes:

- Embedding Layer
- LSTM Layer
- Dense Output Layer

LSTM enhances the limitations of conventional RNNs by incorporating memory cells and gating mechanisms. These gating mechanisms enable the model to retain crucial information for extended text sequences.

Advantages of LSTM

- Better handling of long-term dependencies
- Improved contextual understanding
- Higher classification accuracy

Limitations of LSTM

- More computationally expensive
- Longer training time

5.3 Model 3: LSTM with Pretrained Word Embeddings

Starting off differently, this version boosted the basic LSTM setup using preloaded word data.

The architecture consists of:

- Pretrained Embedding Layer
- LSTM Layer
- Dense Output Layer

Word2Vec or GloVe - these come already trained - help models grasp how words relate, drawing on vectors built from massive amounts of text. Their understanding grows not from scratch, but through patterns locked in earlier learning phases.

Out of pretrained vectors came the embedding matrix, later slotted into the layer. Because context made more sense, classifying got sharper.

Advantages of Pretrained Embeddings

- Better semantic understanding
- Improved generalization

- Faster convergence during training
- Better handling of unseen words

Limitations

- Higher computational cost
- Requires larger memory usage
- Training time may increase

6. Loss Function and Optimizer

6.1 Loss Function

Binary Cross-Entropy loss was chosen for the task since it's a binary classification problem. The loss function quantifies the discrepancy between the predicted values and the actual labels.

6.2 Optimizer

The Adam optimizer was used for training the models.

Adam combines:

- Momentum Optimization
- RMSProp Optimization

It helps improve convergence speed and training stability.

7. Hyperparameters

$\Rightarrow$ The following hyperparameters were used during model training:

Hyperparameter Value:

- Vocabulary Size = 10000
- Embedding Dimension = 50
- Batch Size = 64
- Epochs= 10–20
- Learning Rate = 0.001
- Padding Length = 100

⇒ Early stopping was also used to prevent overfitting.

8. Experiments and Results

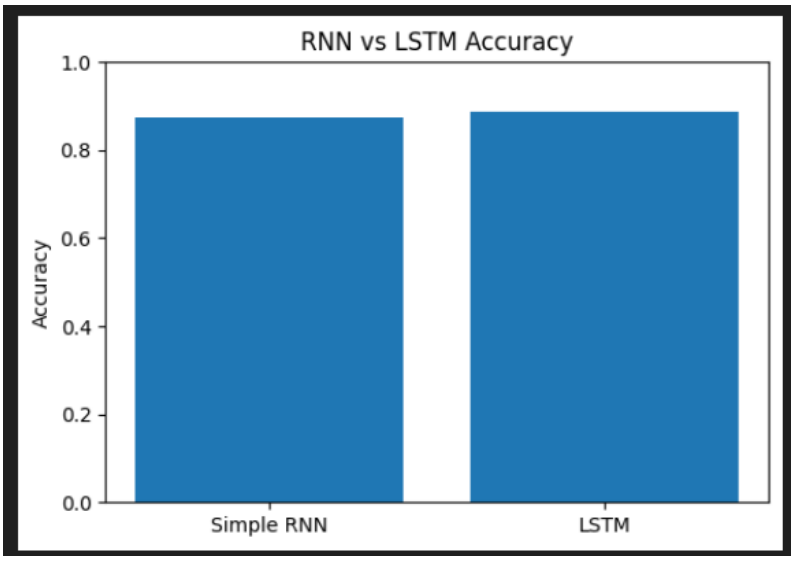
Various experiments were conducted to compare the performance of Simple RNN, LSTM, and LSTM models pretrained with embeddings.

The models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- Training Loss
- Validation Loss

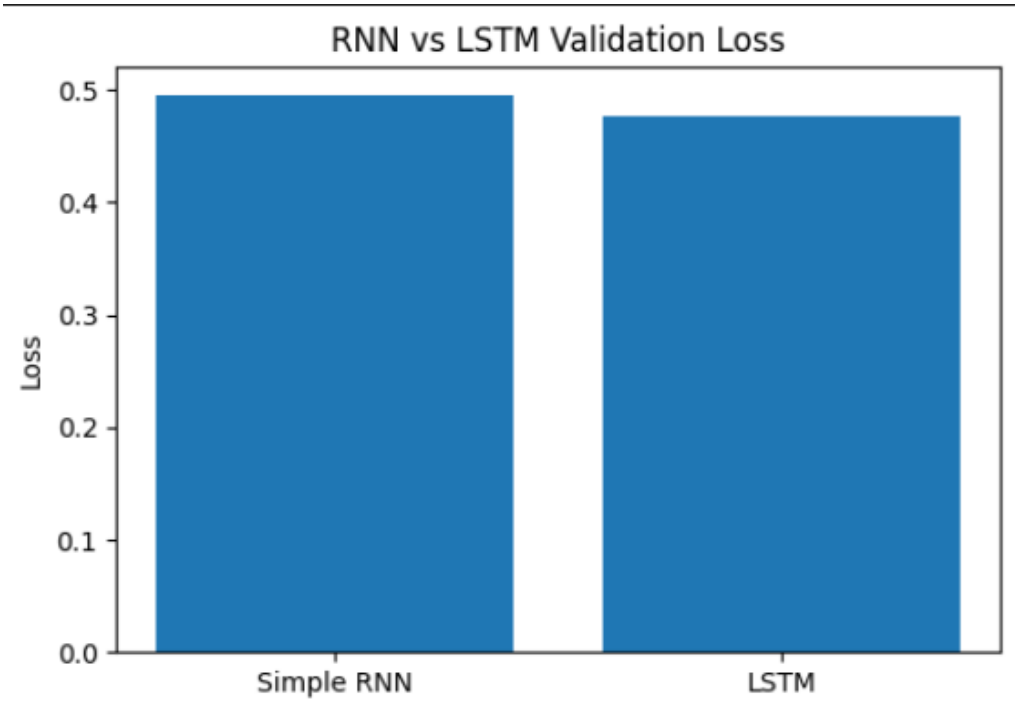
8.1 Training and Validation Accuracy

The LSTM model outperformed the Simple RNN model in terms of validation accuracy. The RNN model exhibited unstable performance and struggled to generalize effectively on unseen data.



8.2 Training and Validation Loss

- The training loss gradually decreased during model training. The LSTM model demonstrated lower validation loss compared to the RNN model, suggesting enhanced learning capabilities.



9. RNN vs LSTM Performance

- The performance comparison between simple RNN and LSTM models showed that LSTM significantly outperformed the Simple RNN model.

```
...          Model  Accuracy  Validation Loss
    0  Simple RNN    0.8840          0.3851
    1        LSTM    0.8814          0.4013
```

Discussion:

Though short on context handling, the basic RNN showed weaker results in labeling tasks. On the flip side, LSTM used special memory units and control gates to grasp deeper patterns across time. Thanks to built-in structures that manage flow of info, it made sense of sequences far better. Pretrained word representations gave it a head start, bringing rich language traits without fresh training. These ready-made features lifted both precision and reliability during testing. Even though learning took longer per cycle, gains in accuracy stayed clear throughout evaluations.

10. Model Evaluation

10.1 Accuracy

- Accuracy measures the overall correctness of the model predictions.

10.2 Precision

- Precision measures how many predicted positive samples are actually positive.

10.3 Recall

- Recall measures how many actual positive samples were correctly identified.

10.4 F1-Score

- F1-score is the harmonic mean of precision and recall.

10.5 Confusion Matrix

The confusion matrix was used to analyze:

- True Positives
- True Negatives
- False Positives
- False Negatives

The LSTM model achieved better balance between precision and recall compared to the RNN model.

- **Model 1 (Simple RNN)**

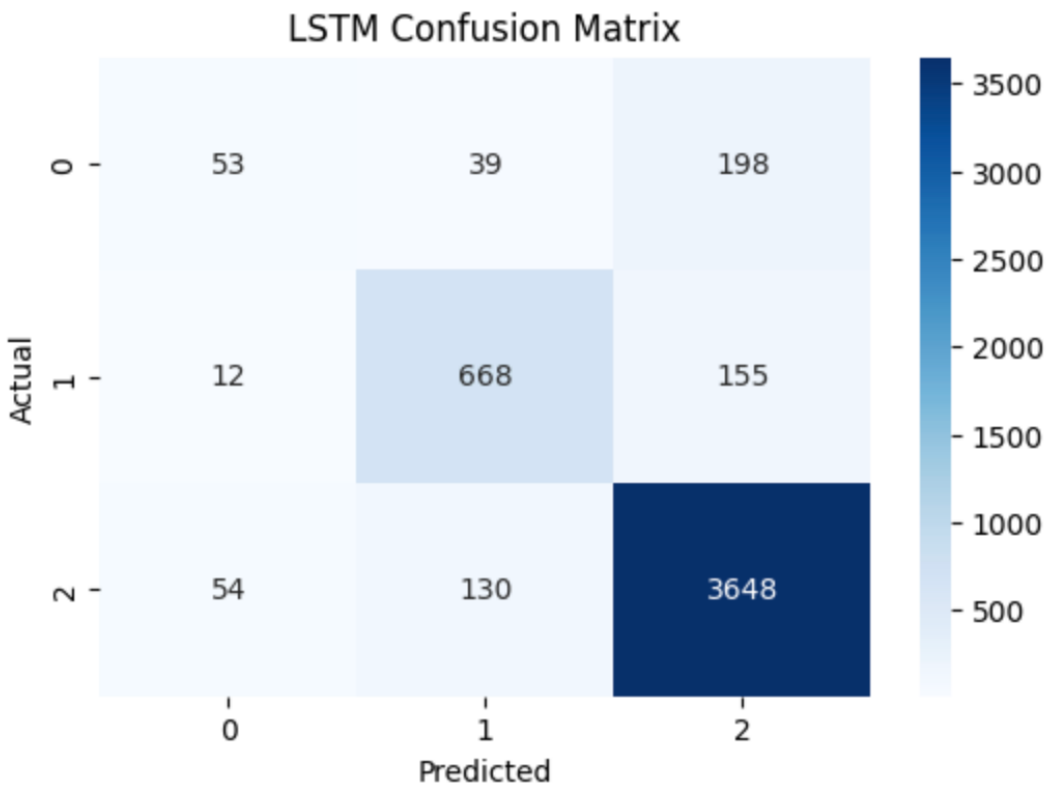
```
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step
LSTM Accuracy: 0.8813798668549526

Classification Report:

              precision    recall  f1-score   support

           0       0.45      0.18      0.26       290
           1       0.80      0.80      0.80       835
           2       0.91      0.95      0.93      3832

    accuracy                           0.88      4957
   macro avg       0.72      0.64      0.66      4957
weighted avg       0.87      0.88      0.87      4957
```



- **Model 2 (LSTM)**

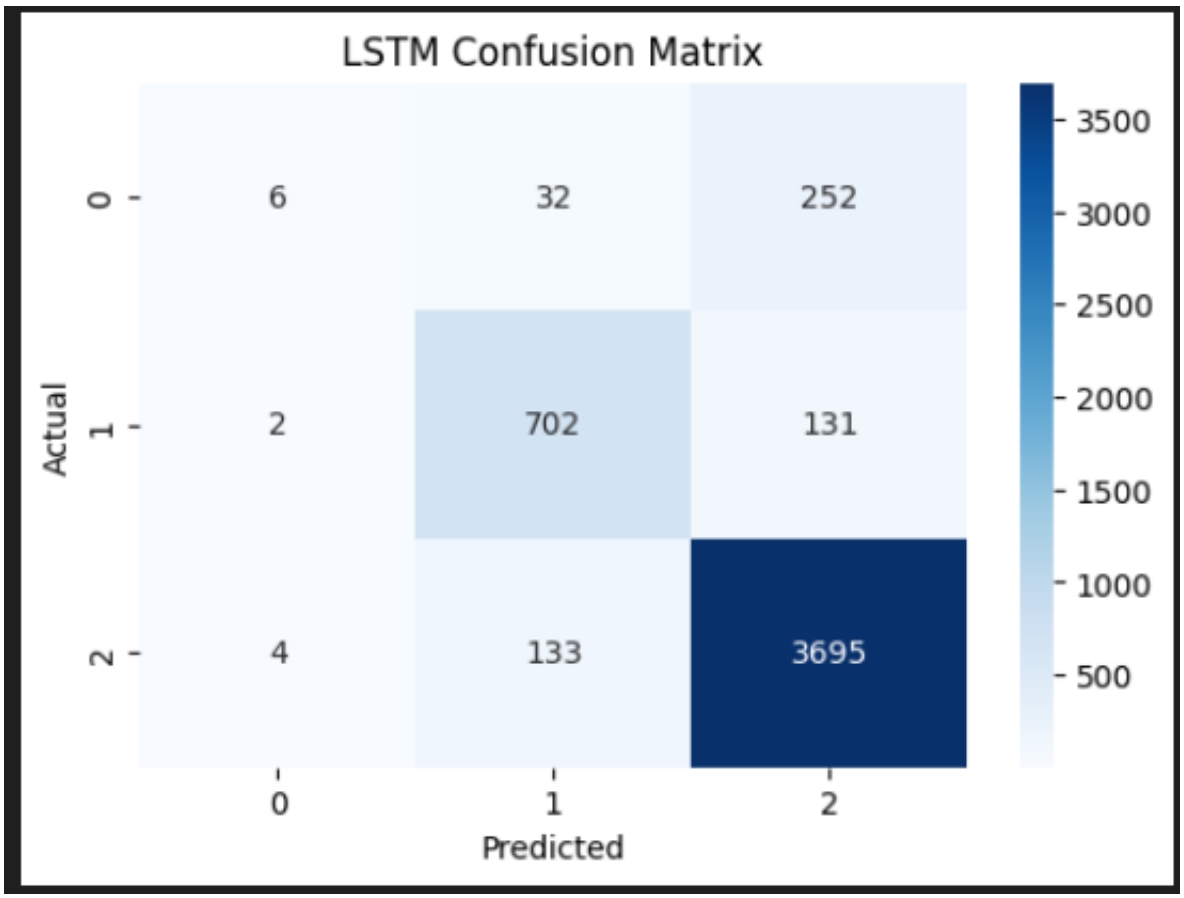
```
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step
Word2Vec Accuracy: 0.8838006858987291

Classification Report:

              precision    recall  f1-score   support

           0       0.59      0.12      0.21       290
           1       0.78      0.82      0.80       835
           2       0.91      0.96      0.93      3832

    accuracy                           0.88      4957
   macro avg       0.76      0.63      0.65      4957
weighted avg       0.87      0.88      0.87      4957
```



- **Model 3 (LSTM with pretrained Word2Vec embeddings)**

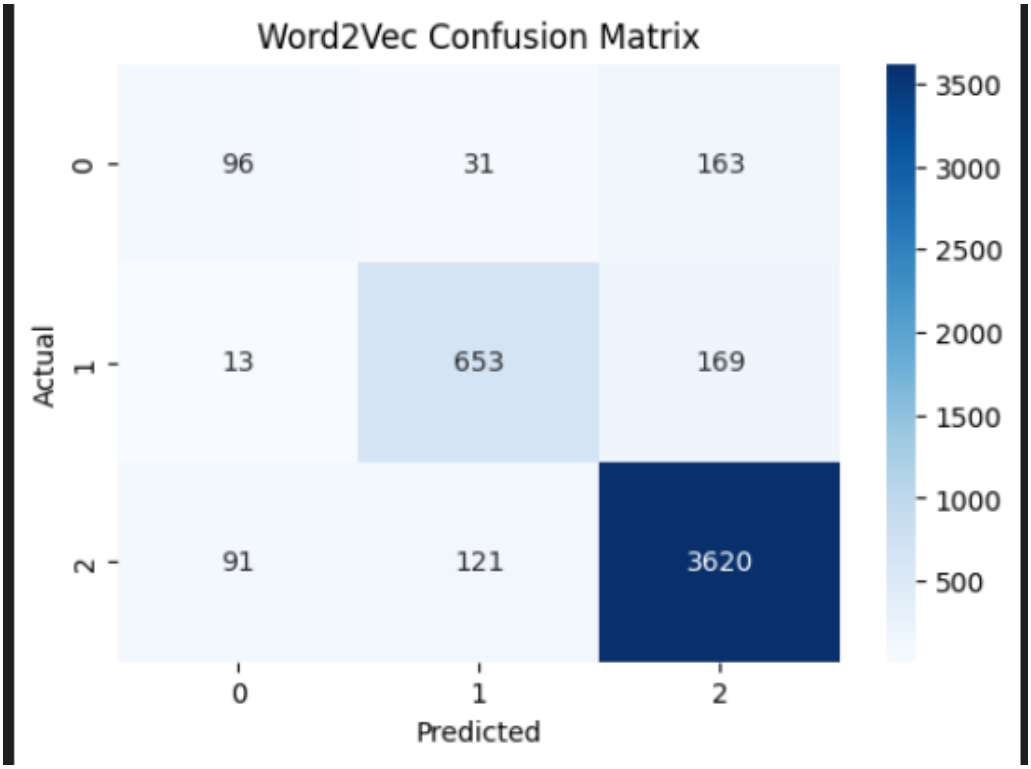
```
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
Word2Vec Accuracy: 0.8813798668549526

Classification Report:

              precision    recall  f1-score   support

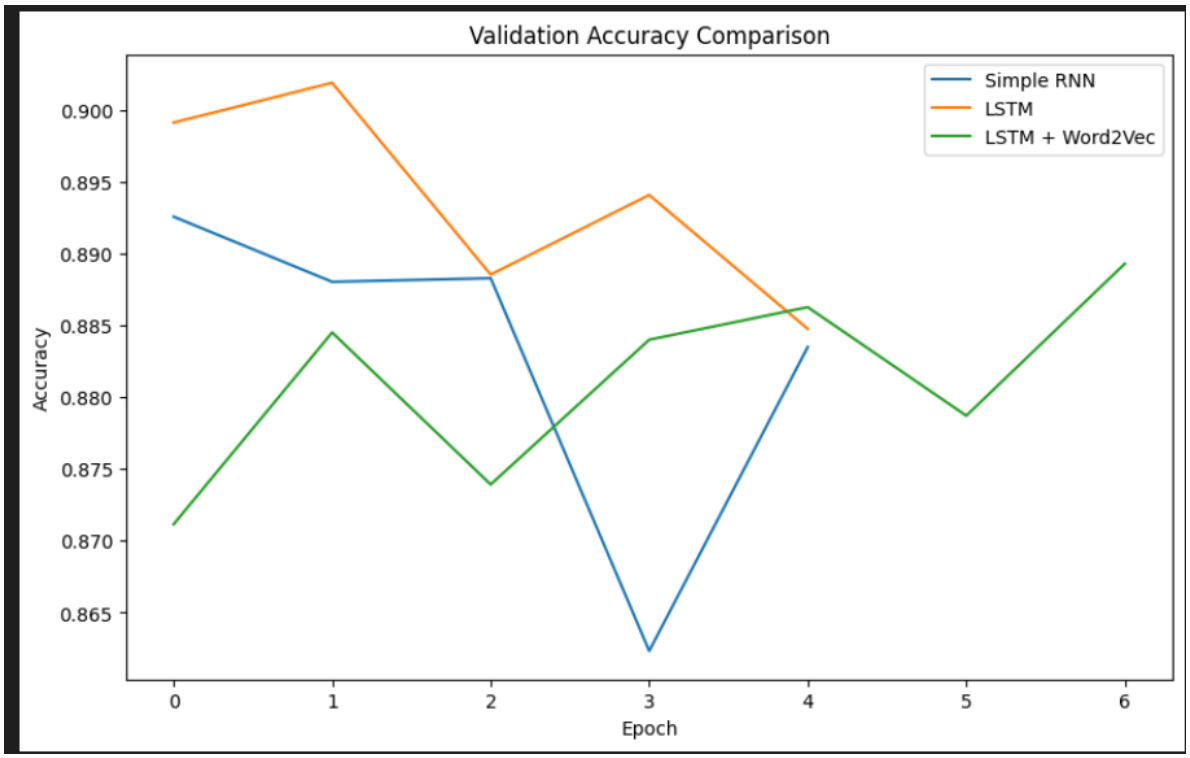
           0       0.48      0.33      0.39       290
           1       0.81      0.78      0.80       835
           2       0.92      0.94      0.93      3832

    accuracy                           0.88      4957
   macro avg       0.74      0.69      0.71      4957
weighted avg       0.87      0.88      0.88      4957
```

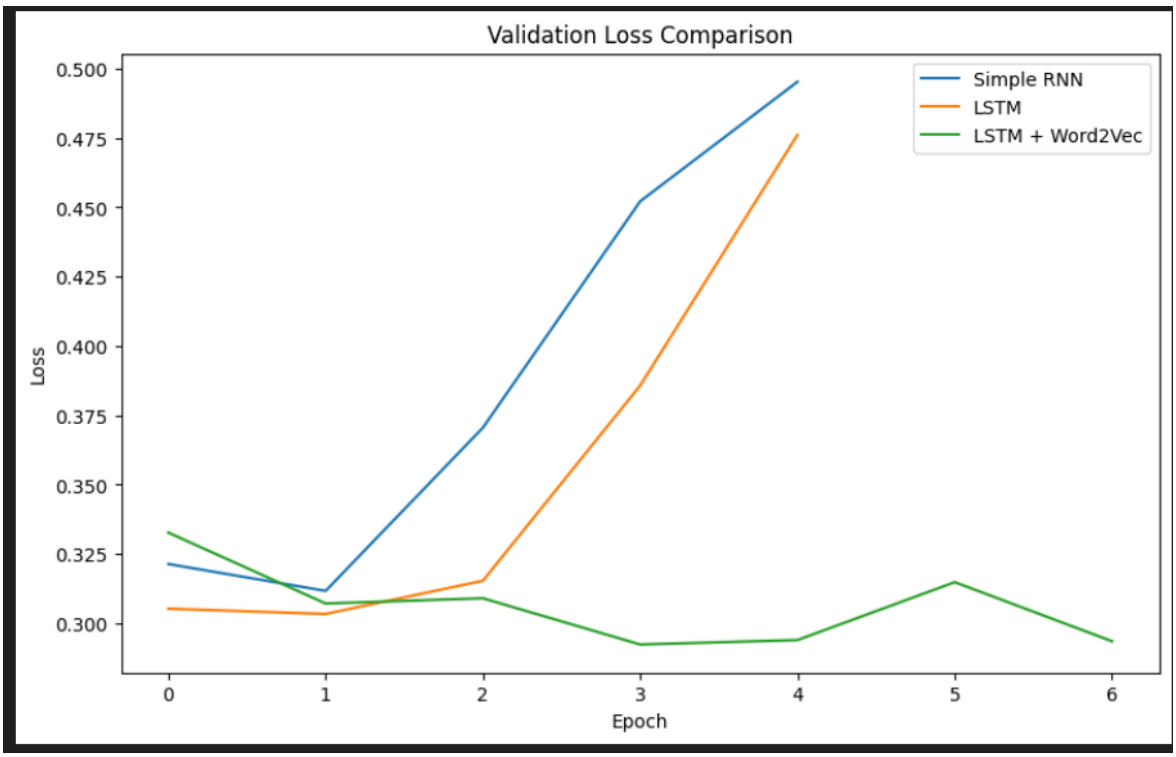


11.Final Comparision of All Models

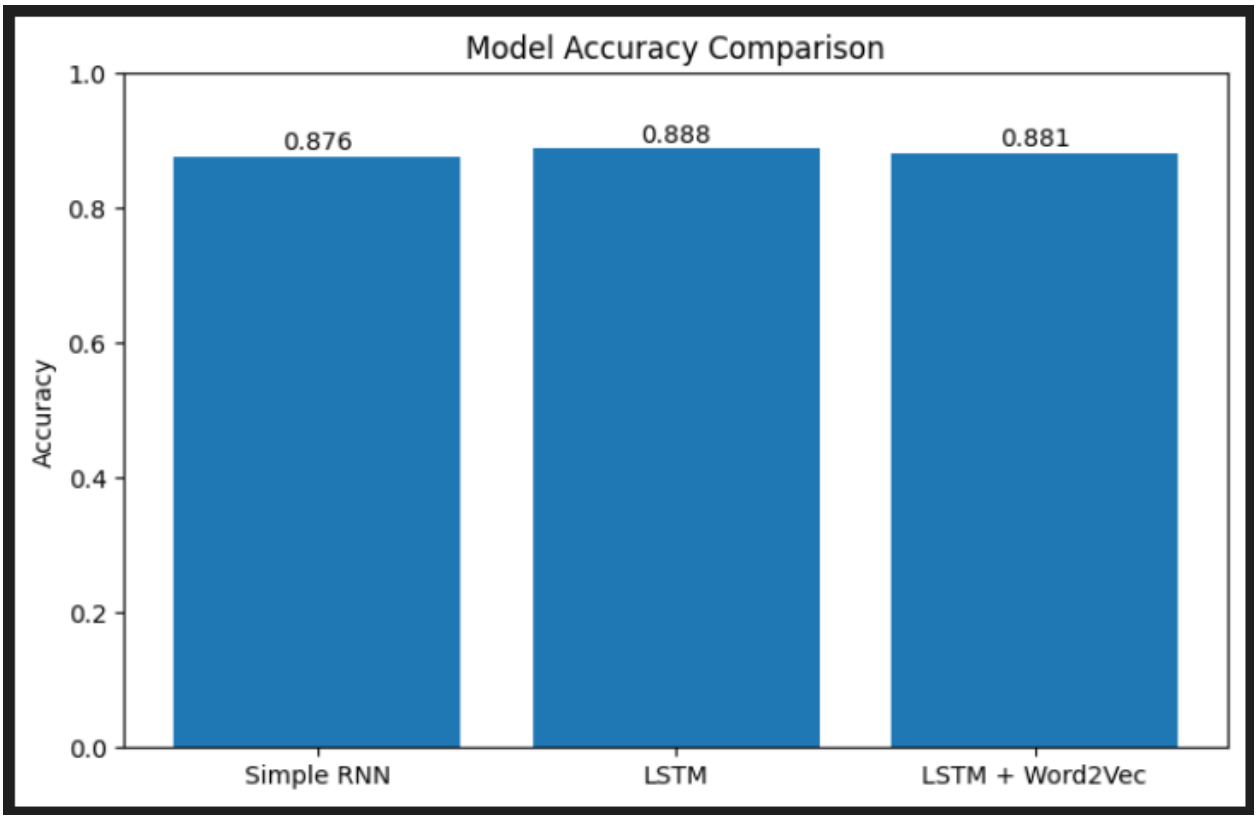
- **Validation Accuracy Comparision**



- **Validation Loss Comparision**



- **Model Accuracy Comparision**



12. Conclusion and Future Work

This project successfully implemented NLP-based hate speech detection using deep learning techniques. Different preprocessing techniques and neural network architectures were applied to classify textual data.

The experimental results showed that:

- LSTM performed better than Simple RNN
- LSTM with pretrained embeddings achieved the best contextual understanding
- Proper preprocessing improved classification performance
- Deep learning models can effectively identify harmful textual content

12.1 Key Findings

Simple RNN

- Faster training time
- Lower classification accuracy
- Difficulty handling long text dependencies

LSTM

- Better contextual understanding
- Higher accuracy and F1-score
- More stable performance

LSTM with Pretrained Embeddings

- Improved semantic understanding
- Better generalization performance
- Higher prediction stability
- More effective for complex textual patterns

12.2 Limitations

The project still has several limitations:

- Limited dataset size
- Possible model overfitting
- Difficulty understanding sarcasm and hidden meaning
- Computational cost of deep learning models

12.3 Future Improvements

Future work may include:

- Using larger datasets
- Hyperparameter tuning
- Applying regularization techniques
- Using pretrained embeddings such as GloVe or Word2Vec
- Exploring transformer-based architectures such as BERT, RoBERTa, and DistilBERT

These improvements can further increase model performance and robustness.

13. Appendix

Viva signed plagiarism report by **Suren Tamang**.

